MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1, 2025/2026

WEEK 7: PLC INTERFACING WITH MICROCONTROLLER AND PC OVER ETHERNET/IP (VER 2.0)

SECTION 1

GROUP 20

LECTURER: DR. WAHJU SEDIONO

DATE OF EXPERIMENT: Monday, 17th November 2025

DATE OF SUBMISSION: Monday, 24th November 2025

PREPARED BY:

NAME	MATRIC NUMBER
ALI RIDHA BIN MUHAMMAD AMINUDIN	2316137
SITI NORATHIRAH BINTI RAZALLY	2319788
NUR ALYA SAKINAH BINTI MOHD MOKHTAR	2319444

ABSTRACT

This experiment explored how to interface a PLC with a microcontroller and PC using the Ethernet/IP protocol. We used the OpenPLC Editor to design and simulate our ladder logic, ensuring it worked virtually before uploading it to an Arduino board for real-time control. The practical side of the project involved building basic circuits, specifically an LED blinker and a start-stop system, which required us to map variables, configure hardware pins, and verify the actual execution. This process gave us valuable hands-on experience in PLC programming and showed us exactly how software simulation translates to physical hardware. Ultimately, it demonstrated that combining PLCs with microcontrollers is a powerful method for industrial automation, while reinforcing that successful integration relies on precise addressing, clean wiring, and careful testing.

TABLE OF CONTENTS

1.0 INTRODUCTION

In the industrial world, PLCs are usually the default choice for running machinery because they are incredibly reliable and handle real-time tasks well. But as technology moves forward, there is a growing trend to pair these systems with microcontrollers and computers using standard protocols like Ethernet/IP. This shift allows for much better communication and control flexibility. We explored this concept firsthand in this experiment by using the OpenPLC Editor to create ladder logic diagrams and then implementing them on an Arduino board.

Our main goal was to build and test basic control setups like a blinking LED and a start-stop circuit, starting in a simulation environment before moving to the actual hardware. This step-by-step process helped us really understand how PLC logic is built, the specific way software variables need to be mapped to digital I/O pins, and the difference between a virtual test and real-world behaviour. It also gave us a chance to apply theoretical concepts, such as scan cycles and timer functions, to a physical setup.

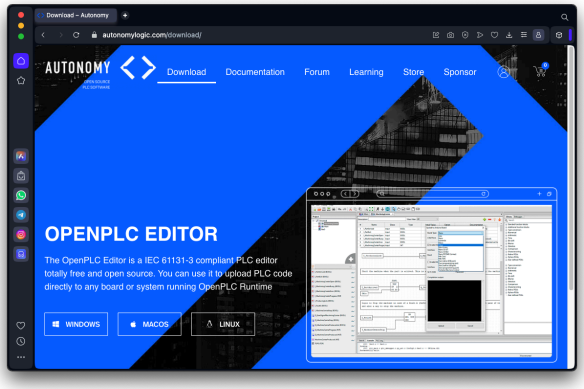
Ultimately, we expect the uploaded programs to execute smoothly on the Arduino, successfully blinking the LED and operating the switch. Getting a positive result here demonstrates that we have a solid handle on interfacing PLCs with microcontrollers, highlighting just how important accurate wiring and pin mapping are for successful real-time control.

2.0 MATERIALS AND EQUIPMENTS

1. OpenPLC Editor software
2. Arduino Board
3. 2 Push Button Switches
4. LED
5. Resistors
6. Jumper Wires
7. Breadboard

3.0 EXPERIMENTAL SETUP

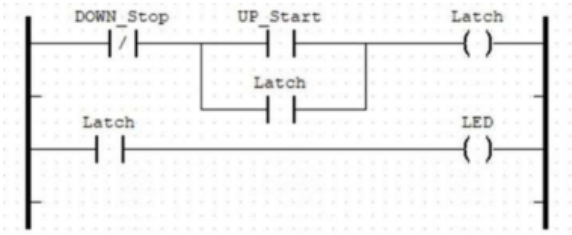
1. Software Setup



1.1 Install and open the OpenPLC Editor on the laptop.

1.2 Create a new project and select Ladder Diagram (LD).

2. Program Creation



#	Name	Class	Type	Location
1	UP_Start	Local	BOOL	%IX0.0
2	DOWN_Stop	Local	BOOL	%IX0.1
3	Latch	Local	BOOL	
4	LED	Local	BOOL	%QX0.0

Documentation
Will Turn ON the LED
Will Turn OFF the LED
Latch Bit Instruction
Output indicator for Latch_Unlatch Program

2.1 Create variables for the LED, Start button, and Stop button.

2.2 Draw the blinking LED ladder diagram.

2.3 Draw the start–stop control ladder diagram.

2.4 Compile both diagrams to check for errors.

3. **Pin Assignment**

3.1 Identify which Arduino pins will be used for input and output.

3.2 Assign each variable to the correct %IX (input) or %QX (output) address in OpenPLC.

4. **Hardware Connection**

4.1 Connect the Arduino to the laptop using a USB cable.

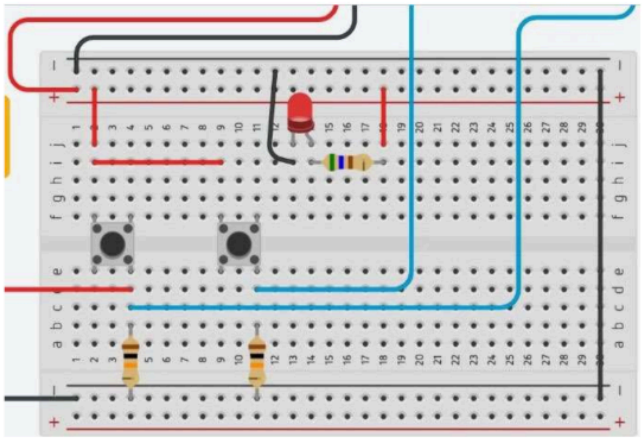
4.2 Select the correct Arduino COM port in OpenPLC.

5. **Program Upload**

5.1 Recompile the program.

5.2 Upload the ladder diagram to the Arduino through the “Transfer to PLC” option.

6. **Circuit Assembly**

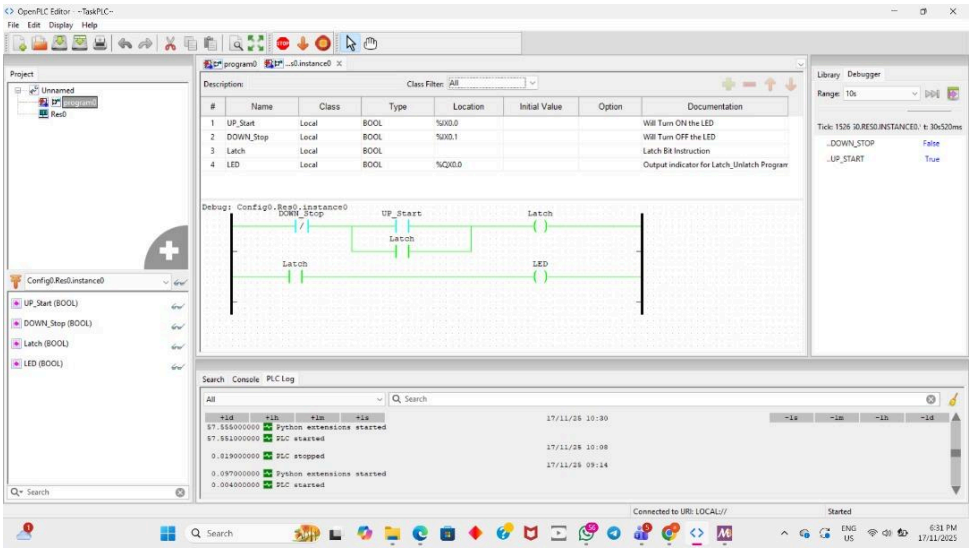


6.1 Place the LED, push buttons, resistors, and jumper wires on the breadboard.

6.2 Connect the LED to the output pin.

6.3 Connect the start and stop buttons to the input pins.

4.0 METHODOLOGY



4.1 Ladder Diagram Development

- The Start–Stop control circuit was designed in the OpenPLC Editor
- All necessary Boolean variables were created and named according to their functions
- The ladder diagram was constructed using a normally open Start contact, normally closed Stop contact, a seal-in contact, and an output coil
- This formed the complete logical structure required for the Start–Stop operation

4.2 Simulation and Verification

- The ladder diagram was compiled to confirm that no errors were present
- The program was simulated within OpenPLC to observe the logical behaviour
- The Start and Stop buttons were tested virtually to ensure the LED output activated, latched, and deactivated correctly according to the design

4.3 Hardware Configuration and Program Upload

- The Arduino board was set as the runtime hardware for executing the PLC logic
- Each variable was assigned a physical address based on OpenPLC's Arduino pin-mapping reference
- The correct COM port was identified, and the ladder program was uploaded to the Arduino via the OpenPLC transfer tool
- Successful upload indicated that the Arduino was running the intended control logic

4.4 Circuit Assembly and Testing

- The physical Start–Stop circuit was assembled on a breadboard using two push buttons, an LED, resistors, and jumper wires
- Each component was connected to the corresponding Arduino pins according to the assigned addresses
- The system was powered and tested to confirm proper functionality:
    - Pressing Start turned on and latched the LED
    - Pressing Stop turned the LED off
- Any wiring or pin-mapping issues were corrected during the testing phase

5.0 DATA COLLECTION

Test Scenario	Start Button	Stop Button	Output (LEC)
Press start only	Pressed	Released	ON
Hold Start	Held	Released	ON
Release start	Released	Released	ON
Press stop	Released	Pressed	OFF
Press start again	Pressed	Released	ON

6.0 DATA ANALYSIS

The primary goal of this analysis was to determine if the ladder logic functioned on the hardware exactly as it did in the OpenPLC software. Regarding the blinking LED task, the timer block was configured with a 1000 ms delay to achieve a target frequency of 0.5 Hz.

$$f = \frac{1}{T_{ON} + T_{OFF}} = \frac{1}{1s + 1s} = 0.5 \text{ Hz}$$

Testing revealed that the LED completed a full ON-OFF cycle every two seconds, confirming that the timer executed correctly and the Arduino responded accurately to the programmed logic.

The start-stop control circuit also behaved as intended. Pressing the Start button activated the LED and kept it latched even after the button was released, while the Stop button immediately cut the signal. This performance indicated that the latch logic and the normally closed stop contact were functioning properly. Furthermore, the hardware detected input changes without delay or instability, suggesting that the wiring and input configuration were sound.

A comparison between the initial simulation and the physical output showed consistent behaviour in both environments. This alignment validates the correctness of the pin assignments, the scan cycle, and the overall logic execution. Ultimately, the data confirms that PLC logic designed in OpenPLC can be effectively implemented on a microcontroller, demonstrating successful system integration and satisfying the experiment's objectives.

7.0 RESULT

The modified ladder diagram successfully created a blinking LED with an increased delay using the timer block. After inserting the timer (TON) into the rung and setting the expression T=1000 m**s**, the LED on the Arduino board blinked at a noticeably slower rate compared to the original configuration. The simulation in the OpenPLC Editor showed the timer counting up before energizing the output coil, and this behavior matched the actual hardware output. Uploading the program to the Arduino produced stable and consistent LED blinking intervals according to the delay value set in the experiment.

8.0 DISCUSSION

This experiment demonstrates how timer blocks can be integrated into ladder logic to control the timing of output devices. By using the standard timer function provided in the OpenPLC Editor, the blinking interval of the LED was successfully modified. The TON timer allows the output coil to energize only after the preset delay has elapsed, which mirrors how real industrial PLCs manage time-dependent processes. The smooth transition between simulation and hardware execution indicates that the OpenPLC environment handles timer functions reliably on an Arduino microcontroller. This experiment also highlights the importance of assigning correct variable locations, ensuring proper physical addressing, and verifying the logic through simulation before transferring to hardware.

9.0 CONCLUSION

In conclusion, this experiment successfully demonstrated the use of a timer block to control the blinking interval of an LED in a ladder diagram. The OpenPLC Editor allowed the timer function to be implemented easily, and the results on the Arduino hardware matched the expected behaviour from the simulation. This confirms that timer-based logic can be effectively applied to microcontroller-based PLC systems, reinforcing an essential skill in industrial automation, which is the ability to manipulate timing parameters within ladder logic to achieve desired control outcomes.

10.0 RECOMMENDATIONS

For future iterations of this experiment, several improvements can be made to enhance both accuracy and learning outcomes. One key recommendation is to incorporate hardware debouncing or software-based filtering for the push buttons. While the inputs in this experiment were stable, real industrial systems often encounter signal noise, and adding debounce logic would give students more realistic exposure to practical control challenges. Additionally, using external power supplies instead of relying solely on USB power could improve system stability, especially if more components are added in future experiments.

Another useful improvement would be to explore more advanced ladder logic elements, such as counters, rising-edge triggers, or interlocks. This would help students extend their understanding beyond basic timers and latches. Including an Ethernet/IP communication test, such as monitoring Arduino outputs from a PC dashboard, would also strengthen the objective of learning PLC networking and make the experiment more aligned with real automation environments.

From a learning perspective, students would benefit from documenting common troubleshooting steps, such as how to correct pin mapping errors, fix compilation issues, or diagnose unresponsive inputs. Many learners struggle with these early in the process, so providing a troubleshooting guide could save time and improve confidence. Finally, future students are encouraged to double-check wiring before uploading programs, test logic in simulation first, and take note of how PLC scan cycles influence the timing of outputs. These insights help build a deeper understanding of how software logic translates into real hardware behaviour, ultimately strengthening their foundation in PLC–microcontroller integration.

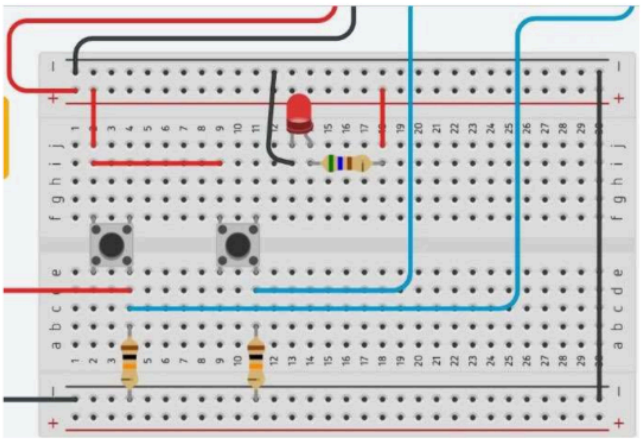
11.0 REFERENCES

Autonomy – Open-source PLC Software. (n.d.). https://autonomylogic.com/

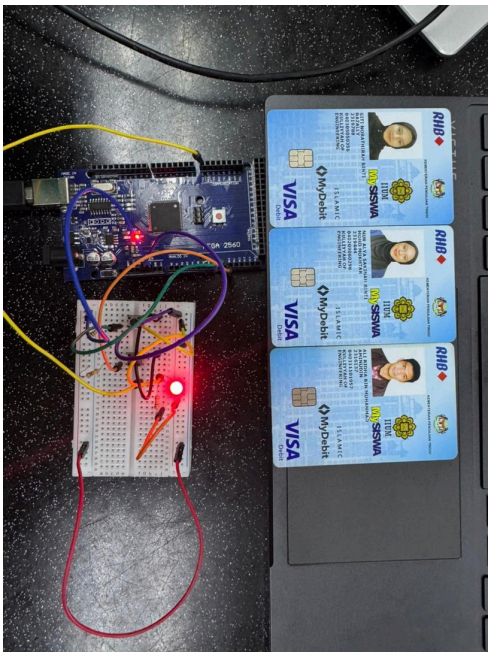
3.2 Creating Your First Project on OpenPLC Editor – Autonomy. (n.d.). https://autonomylogic.com/docs/3-2-creating-your-first-project-on-openplc-editor/

Arshad, B. (2025, January 12). Different types of relays and their applications. Engineers Guidebook. https://engineersguidebook.com/different-types-of-relays-their-application/

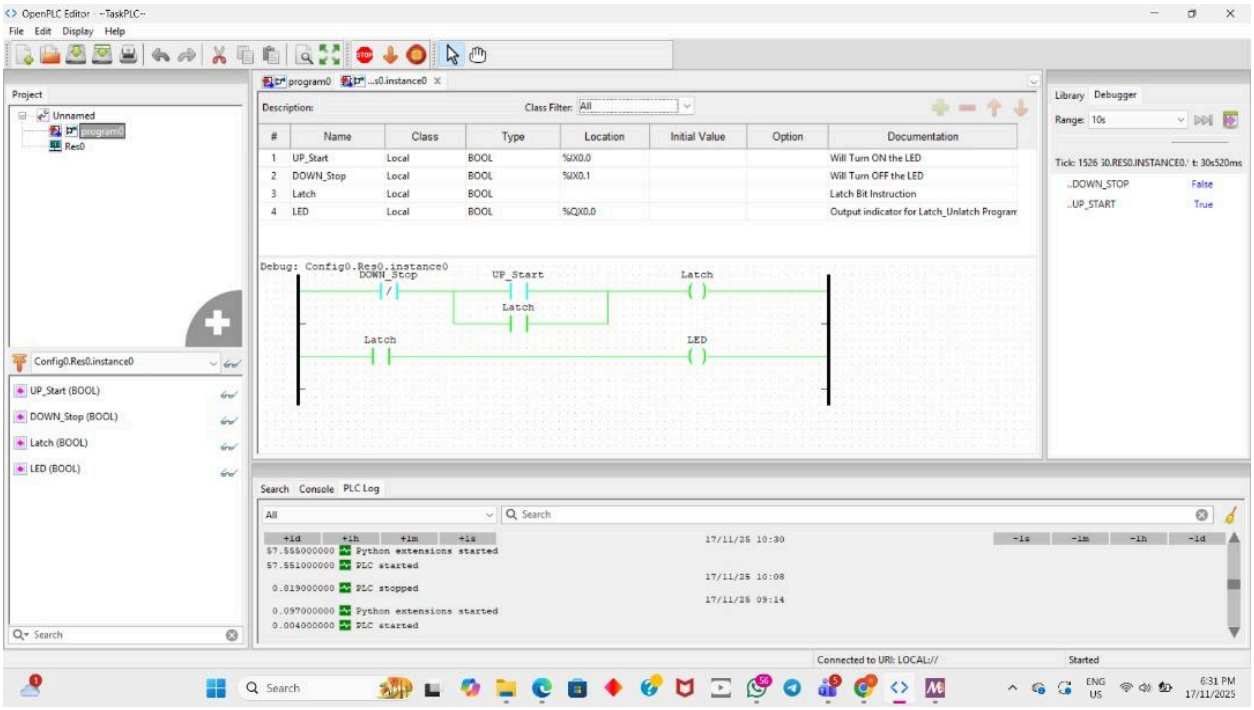
12.0 APPENDICES



(Wiring Diagram for Experiment 7)



(Experiment 7)


OpenPLC Editor - -TaskPLC--
File Edit Display Help
Project
Unnamed
program0
Res0
Config0.Res0.instance0
UP_Start (BOOL)
DOWN_Stop (BOOL)
Latch (BOOL)
LED (BOOL)
Search
program0 ...s0.instance0
Description:
Class Filter: All
Name Class Type Location Initial Value Option Documentation
1 UP_Start Local BOOL %IX0.0 Will Turn ON the LED
2 DOWN_Stop Local BOOL %IX0.1 Will Turn OFF the LED
3 Latch Local BOOL Latch Bit Instruction
4 LED Local BOOL %QX0.0 Output indicator for Latch_Unlatch Program
Debug: Config0.Res0.instance0
DOWN_Stop
UP_Start
Latch
Latch
Latch
LED
Library Debugger
Range: 10s
Tick: 1526 30.RES0.INSTANCE0.' t: 30s520ms
..DOWN_STOP False
..UP_START True
Search Console PLC Log
All
Search
17/11/25 10:30
Python extensions started
PLC started
17/11/25 10:08
PLC stopped
17/11/25 09:14
Python extensions started
PLC started
Connected to URI: LOCAL://
Started
Search
ENG US
6:31 PM
17/11/2025

OpenPLC

ACKNOWLEDGEMENTS

Special thanks to Dr. Wahju Sediono for his guidance and support during this experiment. We would also like to extend our gratitude to our instructors, peers, and the laboratory staff for their valuable insights and assistance throughout the experiment. Their feedback and encouragement have greatly contributed to the success of this project. Finally, we appreciate the resources and learning materials provided, which enabled us to understand and implement serial communication effectively.

Certificate of Originality and Authenticity

We hereby certify that we are responsible for the work presented in this report. The content is our original work, except where proper references and acknowledgements are made. We confirm that no part of this report has been completed by anyone not listed as a contributor. We also certify that this report is the result of group collaboration and not the effort of a single individual. The level of contribution by each member is stated in this certificate. Furthermore, we have read and understood the entire report, and we agree that no further revisions are required. We collectively approve this final report for submission and confirm that it has been reviewed and verified by all group members.

Signature:  Read [/]

Name: ALI RIDHA BIN MUHAMMAD AMINUDIN Understand [/]

Matric Number: 2316137 Agree [/]

Signature:  Read [/]

Name: SITI NORATHIRAH BINTI RAZALLY Understand [/]

Matric Number: 2319788 Agree [/]

Signature:  Read [/]

Name: NUR ALYA SAKINAH BINTI MOHD MOKHTAR Understand [/]

Matric Number: 2319444 Agree [/]